Lab RAG: debugging del retrieval

Modulo 3 · Lezione 6



Il caso comune: il chunk giusto non è nel top-k

Due domande sulla baseline di `rag.py`: semantico, k 4. In tutte e due il chunk con la risposta esiste, ma non è tra i quattro che entrano nel prompt.

Entro quanti giorni posso restituire una scrivania comprata in saldo?

La risposta è nella policy: 14 giorni con un codice promozionale. Quel chunk è **5°**; nel prompt entrano i primi 4, tra cui quello dei 30 giorni del reso standard.

`→ Hai 30 giorni di tempo dalla consegna… [policy-resi-garanzia-001]`

Risposta sbagliata, detta con sicurezza. (non è tra le tredici del set: è una domanda come quelle che scriverete voi)

Cosa è successo nel ticket TCK-1152?

Il ticket esiste, ma il suo chunk è **10°**: nel prompt entrano quattro ticket che parlano d'altro.

`→ Non lo trovo nei documenti.`

Rifiuto.

> Stesso failure, due sintomi. La domanda di oggi: **perché non c'è**, e quale leva lo porta dentro senza rompere il resto.



Agenda

	Blocco	Dove	Tempo
	Brief: tre domande, sei casi, le leve per caso	slide	25′
	Lab 1: il banco di prova	`working/debug.py` · TODO 1–3	70′
	Pausa	—	15′
	Lab 2: caccia ai failure	`working/debug.py` · `LE_MIE_QUERY` · `griglia.md`	55′
	Lab 3: cura mirata	`working/debug.py` · `CONFIG`	75′

Tre blocchi pratici: si scrive lo strumento, lo si usa per trovare i failure, si cerca la cura. Un file di lavoro, `working/debug.py`, dalla cartella `scripts/` (`python working/debug.py`), e una scheda da compilare, `working/griglia.md`. `rerank.py` della lezione scorsa è in `aikit/` con `rag.py` e `hybrid.py`: sono moduli che il file importa, non si toccano.



Tre domande, in ordine

Per ogni domanda che fallisce. Al primo «no» ci si ferma: il colpevole è il passaggio che doveva dire «sì».

1. Il chunk giusto **esiste** nel corpus?
   - no → nessun colpevole: la domanda non ha risposta, e il sistema deve dirlo
2. È nel **top-k** del retrieval, cioè tra i k che entrano nel prompt?
   - no → il retrieval. **Il caso comune.**
3. La risposta **lo usa**?
   - no → la generazione. Raro: se è nel prompt, di solito lo usa.



Come si riconosce il chunk giusto: la frase attesa

Per la domanda 1 serve sapere qual è il chunk giusto. Non si usa il suo id, che cambia se cambiate il chunk size: si sceglie una **frase che sta solo in quel chunk** e la si cerca nel testo di tutti i chunk.

Entro quanti giorni posso restituire una scrivania comprata in saldo?	→	`"ESTATE15"`	→	il codice promozionale che la policy cita: sta in un chunk solo, `policy-resi-garanzia-007`
Posso pagare alla consegna, in contrassegno?	→	`None`	→	nessuna frase da cercare: la risposta non c'è. Giusto se il sistema dice «Non lo trovo nei documenti.»

Per le tredici domande del set la frase attesa è già scritta nel file. Per le domande vostre la scegliete voi, copiandola dal documento.



Non è nel top-k: quanto è lontano?

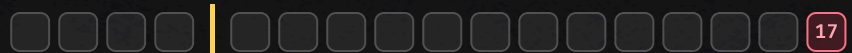
Il retrieval ne recupera k e k entrano nel prompt: per la pipeline, fuori è fuori. Per la diagnosi si guarda più in giù — i primi 10 (`CONFIG["candidati"]`), o 30 — e si vede **dove sta**: decide la cura. Sono due chiavi distinte di `CONFIG`: k è quanto entra nel prompt, `candidati` quanto vede la diagnostica. Alzare `candidati` non cambia la risposta del modello, solo quanto lontano riesci a guardare.

Vicino al taglio



Entro i 10 guardati: il saldo 5°, TCK-1152 10°. Lo portano dentro un k più alto, il `rerank` (riordina i candidati prima del taglio), l'`hybrid` (le parole esatte, come un codice).

Lontano



Oltre i 10 (per vederlo si alza `candidati` a 30): «La scrivania non sta ferma, si muove tutta quando scrivo» — il chunk del manuale dice «oscilla alla massima altezza» ed è 17°. Non c'è un codice, è una parafrasi: l'hybrid non aiuta, e nessuna leva di oggi lo porta nel top-4.

I sei casi

caso	come si riconosce	esempio
A	fuori dal top-k, **vicino**: entro i 10 candidati guardati — il più comune	il saldo 5° · TCK-1152 10°
B	fuori dal top-k, **lontano**: oltre i 10	la scrivania 17ª
C	non esiste nel corpus, e il modello lo dice — non è un failure	«Posso pagare in contrassegno?» → «Non lo trovo»
D	non esiste, e il modello risponde lo stesso raro	«Come si toglie il blocco tastiera?» → procedura inventata (il manuale dice solo come si attiva)
E	nel prompt, ma la risposta lo ignora raro	«Cosa copre SRV-02?» → chunk 3°, nel prompt, «Non lo trovo»
F	risposta giusta, ma costosa — non si incontra cacciando: si produce nel Lab 3	k 20: tutto giusto, quattro volte i token

E · PERCHÉ UN CHUNK NEL PROMPT VIENE IGNORATO

Due cose insieme. La riga che risponde è una delle tre di un elenco (SRV-01, SRV-02, SRV-03), dentro il chunk del listino. E sopra, nel prompt, ci sono due chunk di altri documenti sugli altri due servizi. Il modello non riconosce quella riga come la risposta, e rifiuta: si vede leggendo la risposta, non le posizioni.

F · DOVE SI INCONTRA

A k 4 una risposta giusta è giusta e basta: il costo si vede solo alzando k, ed è quello che si fa nel Lab 3.



Quale leva per quale caso

caso	leve in CONFIG	cosa fanno
A	`k` più alto · `"rerank"` · `"hybrid"`	ne entrano di più · riordina i candidati prima del taglio · cerca le parole esatte e i codici
B	`"hybrid"`, se c'è un codice	altrimenti nessuna leva di oggi: la parafrasi → Modulo 3 · Lezione 13
C	—	niente da curare
D	`soglia` · istruzioni	sotto uno score il chunk non entra (solo col semantico o con rerank Cohere, dove lo score è una pertinenza): se non entra niente, il modello non ha da dove inventare · non dedurre ciò che non c'è
E	`"rerank"` + `k` basso · istruzioni	metterlo primo e togliere i distrattori · le regole del prompt
F	`k` più basso	verificando che la risposta resti giusta

La `soglia` taglia per pertinenza, quindi vale solo dove lo score è una pertinenza: il semantico e il rerank Cohere. Con l'hybrid o il rerank LLM lo score è una posizione, e il file la ignora dicendolo a schermo.

`LA CURA INGENUA, MISURATA`

k 20 fa entrare tutto ciò che è vicino: 12 giuste su 13. Ma il prompt passa da **600 a 2.400 token** su ogni domanda: qui il corpus è di 82 chunk e ne entra un quarto ogni volta. È il caso F.



La regola anti-regressione

Ogni cura si rimisura su **tutto** il query set, non sulla domanda che si voleva curare. Si guarda il numero in fondo, poi riga per riga chi era giusto e non lo è più.

```
# RIASSUNTO · retrieval semantico · k 4
  #  domanda                 pos  top-k  giusta
  1  …ticket TCK-1152?        10  no     no
 12  …si ferma a metà corsa?   1  sì     sì

  risposte giuste: 10/13
```

```
# RIASSUNTO · retrieval hybrid · k 4
  #  domanda                 pos  top-k  giusta
  1  …ticket TCK-1152?         1  sì     sì
 12  …si ferma a metà corsa?   2  sì     sì (2/3)

  risposte giuste: 12/13
```

UNA QUERY CHE PEGGIORA VA SPIEGATA

Con l'hybrid TCK-1152 passa da 10° a 1°, e il set da 10 a 12 giuste. Ma «Perché il piano si ferma sempre a metà corsa?» scende da 1° a 2°, e in un giro su tre la risposta cita l'altro ticket. Si scrive nella colonna regressioni, con il perché.

LA GENERAZIONE OSCILLA DA SOLA

Se una risposta cambia tra un giro e l'altro senza aver toccato niente non è una regressione: si rilancia, e si annota «in N giri su 3».



Cosa non si rifà oggi

NON SI RIFÀ

L'A/B di prompt e il tuning a tentativi di Modulo 2 · Lezione 12. Quelle leve le conoscete: oggi si decide **quando** usarle.

NON SI CURA: LA PARAFRASI

La scrivania, caso B: «non sta ferma» contro «oscilla alla massima altezza». In griglia: «nessuna leva di oggi» (→ Modulo 3 · Lezione 13).

NON SI CURA: IL CORPUS CHE SI CONTRADDICE

«Qual è la portata massima della LD-210?»: `110` kg nel listino, `100` kg nel manuale. Nessuna leva sceglie: il problema è nei dati.



I file della giornata

- `CONFIG` — le leve
- `SOLO` — una coppia (domanda, frase attesa) per provarne una sola
- `QUERY_SET` — le tredici domande, con la frase attesa · `LE_MIE_QUERY` — vuota
- `prepara()` · `cerca()` — date: le leve applicate, il retrieval secondo `CONFIG`
- `TODO 1` · `trova_nel_corpus(frase)` — esiste?
- `TODO 2` · `diagnostica(query, frase_attesa)` — le tre domande su una query
- `TODO 3` · `giro()` — tutto il set, poi la tabella
- `helper di stampa` · `main (SOLO oppure il giro)`
- `working/griglia.md` — la scheda da compilare: i failure, le cure, la CONFIG finale
- `MAPPA.md` — dove sta cosa: i documenti, aikit/, il database

`CONFIG` → COSA MUOVE

`"retrieval"` · `"backend"` · `"chunk_size"` · `"soglia"`
quali chunk, e in che ordine (backend solo con rerank; la soglia toglie i chunk sotto uno score)

`"k"`
quanti entrano nel prompt

`"candidati"`
quanti ne guarda la diagnosi, per dire dove sta il chunk giusto

`"istruzioni"`
le regole che il modello segue con il prompt



Lab 1: il banco di prova

SCRIVETE VOI — da soli, con affiancamento

`working/debug.py` · TODO 1-3 · 70 minuti · consegna in `CONSEGNA.md`

si rientra quando
il giro stampa la tabella

TODO 1 · `trova_nel_corpus(frase)` · **esiste?**

gli id dei chunk che contengono la frase; i chunk sono in `hybrid.INDICE["chunks"]`

TODO 2 · `diagnostica(query, frase_attesa)` · **le tre domande**

la frase nel corpus, la posizione tra i candidati e se è entro k, poi `rag.genera()` sui primi k e se cita il chunk atteso → `(posizione, nel_topk, giusta)`

TODO 3 · `giro()` · **tutto il set**

`diagnostica()` su ogni domanda, poi `stampa_tabella()`

COME SI PROVA

Una domanda alla volta: la coppia in `SOLO`, in cima al file, e il main usa quella al posto del giro. Il TODO 2 è il più lungo: provatelo mentre lo scrivete, senza aspettare di averlo finito.

FATTO QUANDO

Con `SOLO = None` il giro gira su tutto il set e stampa la tabella: **10/13** sulla baseline (9 o 11: la generazione oscilla, si rilancia).

Errori tipici: in `CONSEGNA.md` e in fondo a `README.md`.



Pausa

15 minuti

Al rientro Lab 2: la caccia ai failure, con lo strumento appena scritto.

`working/debug.py · LE_MIE_QUERY`



Lab 2: caccia ai failure

SCRIVETE VOI — le domande, non il codice

`working/debug.py` · `LE_MIE_QUERY` · 55 minuti · si annota in `working/griglia.md`

si rientra con la griglia

1 · LE TRE CHE GIÀ SBAGLIANO

Il giro sulla baseline dà 10/13. Le tre che mancano si classificano per prime: per ognuna, dove si ferma la catena delle tre domande e dove sta il chunk. Una riga ciascuna in `griglia.md`, con la lettera del caso.

2 · LE VOSTRE

Almeno tre domande vostre sui sei documenti — con un codice, con parole diverse dal documento, su cose che non ci sono — in `LE_MIE_QUERY` con la frase attesa, trovata con `trova_nel_corpus()`. Poi scambiatele col vicino: nessuno cerca solo dove sa di trovare.

FATTO QUANDO

La prima tabella di `griglia.md` ha almeno **sei failure** con la lettera del caso, di almeno **due casi** diversi (il caso C si annota, ma non conta). Se una domanda vostra viene risposta bene non è un failure: si scarta e se ne scrive un'altra.

Dopo la caccia: i casi trovati

caso	domanda	cosa succede	
A	Cosa è successo nel ticket TCK-1152?	chunk 10° → «Non lo trovo»	set
A	Entro quanti giorni posso restituire una scrivania comprata in saldo?	chunk 5° → «30 giorni», con la fonte sbagliata	vostra
A	Qual è la versione corrente del firmware della centralina?	chunk 6° → «Non lo trovo»	vostra
B	La scrivania non sta ferma, si muove tutta quando scrivo: che controllo faccio?	chunk 17° → «Non lo trovo»	set
C	Posso pagare alla consegna, in contrassegno?	non esiste → «Non lo trovo nei documenti.» ✔	set
D	Come si toglie il blocco tastiera una volta attivato?	non esiste → procedura inventata, cita il manuale	vostra
E	Cosa copre SRV-02?	chunk 3°, nel prompt → «Non lo trovo» (anche primo, col rerank)	set
E	Posso usare un alimentatore da 4A al posto di quello in dotazione?	chunk 1°, nel prompt → «Non lo trovo»	vostra

«set» = una delle tredici; «vostra» = una domanda scritta da noi, come le vostre. Chi ha lo stesso failure in un caso diverso spiega perché. Poi la pausa; al rientro, le leve.



Lab 3: cura mirata

SCRIVETE VOI — oggi in CONFIG, non nelle funzioni

`working/debug.py` · `CONFIG` · 75 minuti · si annota in `working/griglia.md`

si rientra con la griglia completata

IL CICLO, PER OGNI FAILURE DELLA GRIGLIA

1 · scegli **una** leva coerente con il caso
2 · cambiala in `CONFIG` e rilancia il giro
3 · annota nella seconda tabella «dopo» e le **regressioni**: le query del set che erano giuste e ora no
4 · riporta `CONFIG` alla baseline prima della leva successiva

se una leva da sola non basta, provane una seconda **insieme**: in griglia vanno tutte e due

CASO → LEVE

A	`k` più alto · `"rerank"` · `"hybrid"`
B	`"hybrid"`, se c'è un codice
D	soglia · istruzioni
E	`"rerank"` + `k` basso · istruzioni
F	`k` più basso

FATTO QUANDO

1 · Almeno **due failure** curati (TCK-1152 con hybrid è l'esempio del brief: non conta).
2 · Almeno una cura usa `"hybrid"` o `"rerank"`.
3 · Per ogni query del set che è **peggiorata** sapete dire perché.



«Se finisci prima»: in `CONSEGNA.md`.

Dopo il Lab 3: le leve misurate sul set

leva	giuste su 13	cura	regressioni · costo
`baseline`	10	—	600 token in ingresso a domanda
`"hybrid"`	12	TCK-1152 10° → 1° · versione 6° → 2°	metà corsa 1° → 2° (giusta 2 su 3) · il saldo 5° → 9° · un indice in più
`"rerank" (Cohere / LLM)`	12	il saldo 5° → 2°-3° · versione → 1°	SRV-02 primo, e rifiutato lo stesso · una chiamata in più a domanda
`k = 8`	10	il saldo e la versione entrano	TCK-1152 resta 10° · ×2 token
`k = 20`	12	tutto ciò che è tra i candidati entra · la scrivania 2 su 3	×4 token su ogni domanda
`chunk_size = 300`	9	il saldo 5° → 2°	SRV-02 3° → 6° · TCK-1152 9° · metà corsa 2 su 3 · si ri-indicizza (140 chunk)
`soglia = 0.45`	10	il blocco tastiera rifiuta, 3 su 3	—
`istruzioni (2 varianti)`	10	—	SRV-02 e blocco tastiera invariati
`"rerank" (Cohere) + k = 2`	12	SRV-02 risponde, 3 su 3: via il chunk della garanzia SRV-03	il saldo, 3° con Cohere, esce dal prompt · con l'LLM l'ordine oscilla: serve k 1



Cosa non si cura con nessuna leva

LA PARAFRASI (CASO B)

La scrivania non sta ferma, si muove tutta quando scrivo

17ª nel semantico, 28ª nella ricerca per parole (BM25). Con `candidati = 30` si vede; con il rerank Cohere sale 9ª; nel prompt a k 4 non entra mai. Entra solo con k 20, pagando ×4 su tutte le domande. «Non sta ferma» contro «oscilla alla massima altezza»: la cura è a Modulo 3 · Lezione 13.

IL CORPUS CHE SI CONTRADDICE

Qual è la portata massima della LD-210?

Nel listino `portata 110 kg`, nel manuale `portata massima 100 kg`. In baseline risponde 100 kg dal manuale; con hybrid o rerank il listino entra e risponde 110 kg. Qualunque leva sceglie per caso: il problema è nei dati.

L'ALIMENTATORE DA 4A (CASO E)

Posso usare un alimentatore da 4A?

Caso E. Il ticket TCK-1201 è 1° nel prompt con ogni leva, e il modello rifiuta: il ticket racconta un caso («alimentatore non originale da 4A», «suggerito di non usare alimentatori di terze parti»), non dà una regola, e il modello non deduce. In griglia: «nessuna leva di oggi».

Prima di chiudere, confrontate le griglie: lo stesso failure curato con leve diverse — il saldo con il rerank, con `k = 8` o con `chunk_size = 300` — e chi ha avuto meno regressioni.



La prossima lezione

La tabella di oggi:
`domanda · frase attesa · giusta`

→

Modulo 3 · Lezione 7 — Tool Calling Intro
il modello che chiama funzioni

Più avanti, a Modulo 3 · Lezione 10, la stessa tabella diventa il golden set e le tre domande di oggi diventano metriche.



Grazie

prossima lezione: Modulo 3 · Lezione 7 — Tool Calling Intro

